# Decidable Type-Checking for the Dependent Categorical Combinators

Lexi Aiello[1] and Eric Klavins[2]

[1]Department of Philosophy, University of Washington
[2]Electrical & Computer Engineering Department, University of Washington

April 30, 2026

**Abstract**

We present an extension of the categorical combinators [4] to symmetric monoidal categories, enabling affine dependent types of the nonlinear categorical combinator terms. We formalize the metatheory in the Lean 4 proof assistant. We prove several important properties of the calculus, including that type-checking is decidable, and that the affine restriction of the combinators is strongly-normalizing, and globally confluent. We achieve judgmental decidability in principle by showing that the types of all objects can be expressed using the affine restriction, which we demonstrate is strongly-normalizing by a syntactic decreasing measure. We outline key challenges and architectural strategies for formalizing a dependent combinator calculus with such properties in a proof assistant, and describe our progress towards compiling a $\lambda$-calculus with dependent types ($\Pi$-types and $\Sigma$-types) to our combinators.

## 1 Introduction

Simply-typed combinatory logics like the $SK$-combinators are known to be equivalent to the simply-typed $\lambda$-calculus [1]. These calculi can avoid contexts, and might enjoy benefits as such (e.g., unification). Work has been done on extending these logics with dependent types [2], but no implementation has been proposed, nor have such theories been formalized in a proof assistant.

Combinatory logics suffer from the known disadvantage that they are an inefficient compilation target. For example, when compiling the $\lambda$-calculus to the $SK$-combinators, terms explode in size due to lacking variable substitution [6, p. 267].

Numerous alternative combinatory compilation targets have been proposed. One such calculus is the categorical combinator calculus [4]. The categorical combinators are best described as a literal interpretation of cartesian closed categories as a Lisp-like Debruijn-indexed machine (the Categorical Abstract Machine [3]). Variable substitution is modeled intuitively as composing binary product projections [3, p. 6] over an environment of nested tuples. The categorical combinators have been extended with simple types [4]. There has also been some investigation into extending them to a dependent type theory similar to the Calculus of Constructions [7], but no implementation has been defined.

In this paper, we formalize a metatheory of a dependently-typed categorical combinator calculus with affine types and non-linear terms. We show that type inference is decidable and deterministic. We accomplish this goal in principle by observing that the types of the dependent categorical combinators are encodable with an affine subset of the usual reduction rules. We do not assume $\Pi$-extensionality, but give an $\eta$-reduction algorithm as in Curien's categorical combinators [4, p. 221]. By showing that the affine restriction of the calculus $\eta$-reduction are strongly normalizing through a syntactic decreasing measure, we achieve total affine reduction. We show that the calculus satisfies global confluence by Newman's lemma [5] using our measure. We prove completeness and soundness of $\eta$ and affine $\beta$-reduction with respect to normal forms. We avoid conversion in the typing judgment by eagerly normalizing types, and thus achieve deterministic, decidable type inference. We outline several of our architectural decisions including proof automation with typeclasses critical to satisfying our formalization goals in Lean 4.

# 2 Background

## 2.1 The Untyped Categorical Combinators

The categorical combinators [4] are the basis of the Categorical Abstract Machine [3]. They are a potential compilation target for the $\lambda$-calculus [3]. Categorical combinators simulate variable substitution through projections over nested tuples ("couples", or binary products) constituting an "environment". Binders are simulated through a curry combinator ($\Lambda$) which applies its two arguments as a couple to the body ($\Lambda(x)$). The reduction rules for the calculus resemble the rules of the SK combinator calculus with Debruijn indices, and their categorical semantics are given by cartesian-closed categories [3, p. 178].

$$
\begin{array}{ll}
\text{(ass)} & (x \circ y)z = x(yz) \\
\text{(fst)} & Fst(a, b) = a \\
\text{(snd)} & Snd(a, b) = b \\
\text{(dpair)} & \langle x, y\rangle z = (xz, yz) \\
\text{(curr)} & \Lambda(x)yz = x(y, z) \\
\text{(app)} & App(f, x) = fx \\
\text{(quote)} & `(x)y = x
\end{array}
$$

Figure 1: Reduction rules of the untyped combinators as seen in the Categorical Abstract Machine.

## 2.2 The Simply-Typed Categorical Combinators

Curien extends the combinators with type annotations [4, p. 220], enabling simple typing. Quotation is omitted in the simply-typed variant. The inference rules are as such:

$$
\textsc{Id}\ \frac{\alpha : \text{Type}}{Id(\alpha) : \alpha \to \alpha} \qquad
\textsc{Coup}\ \frac{x : \alpha, y : \beta}{(x, y) : \alpha \times \beta} \qquad
\textsc{Comp}\ \frac{x : \beta \to \gamma, y : \alpha \to \beta}{(x \circ y) : \alpha \to \gamma}
$$

$$
\textsc{Fst}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{Fst(\alpha, \beta) : \alpha \times \beta \to \alpha} \qquad
\textsc{Snd}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{Snd(\alpha, \beta) : \alpha \times \beta \to \beta} \qquad
\textsc{Pair}\ \frac{x : \alpha \to \tau_1, y : \alpha \to \tau_2}{\langle x, y\rangle : \alpha \to \tau_1 \times \tau_2}
$$

$$
\textsc{Curr}\ \frac{x : (\alpha \times \beta) \to \tau}{\Lambda(x) : \alpha \to \beta \to \tau} \qquad
\textsc{Appl}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{App(\alpha, \beta) : ((\alpha \to \beta) \times \alpha) \to \beta} \qquad
\textsc{App}\ \frac{f : \alpha \to \beta x : \alpha}{fx : \beta}
$$

Figure 2: Inference rules of the simply-typed categorical combinators

Curien's categorical combinators are nominally combinatory, but differ from $SK$ and $BCKI$ calculi in that points are still present. Type arguments are given as points, projection mappings in pairs are given as points ($\langle f, g\rangle$), the Curry combinator is defined by a point ($\Lambda(x)$), couple members are specified as points ($(a, b)$), and composition is pointed with respect to $f$ and $g$ ($(f \circ g)x$). This design is quite natural when interpreting projection as corresponding to Debruijn indices. Take an example reduction in the untyped calculus:

$$
\begin{aligned}
&\Lambda(\Lambda(App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle))nfx \\
&= \Lambda(App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle)(n, f)x \\
&= (App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle)((n, f), x) \\
&= App(f, App(App(n, f), x)) \\
&= f(nfx)
\end{aligned} \tag{1}
$$

Despite every visible operation being bound by a point, arguments in scope can be rearranged through projections, composition, pairing, and application (evaluation in the CCC).

# 3 Dependent Categorical Combinators

Our primary insight when extending the categorical combinators to dependent types is that the types of all of the categorical combinators are representable without making use of the pair rule. That is, their types are affine. We refer to the subset of the categorical combinators excluding the pair rule as the "affine restriction." By showing that the affine restriction is strongly normalizing and that reduction is complete, we gain decidable judgmental equality ($\equiv$) and a decision procedure for type-checking.

We interpret $\Sigma$-types as a telescope context as usual. As in the Curry combinator, we specify combinator type arguments as points that internalize an indexing of the environment. For example, the $\Pi$ combinator is inferred like such:

$$\frac{\textsc{Pi}}{\delta : \text{Sort } m, f_\alpha : \alpha \to \text{Sort } n, f_\beta : \Pi(s : \delta)(x : f_\alpha s), \text{Sort } o}{\Pi(f_\alpha, f_\beta) : \delta \to \text{Sort } o} \tag{2}$$

We interpret couples as dependent sums. $\Sigma$-couples make traversing data in a well-typed way quite convenient. All of our objects are well-typed, and $\Pi$ elimination sorts can be computed with no extra assumptions. We define most of the type arguments of the combinators as functions of the environment eliminating into Sort $n$. By specifying type arguments in this way, potential duplication of items in the context can be avoided (e.g., in $\Pi$, where the environment is shared across $f_\alpha$ and $f_\beta$), thus further enabling a purely affine type theory.

## 3.1 $\Sigma$-Couples

Recall the usual typing of a $\Sigma$-pair:

$$\frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n}{\Gamma \vdash \Sigma(\alpha, \beta) : \text{Sort } max(m, n)} \tag{3}$$

A $\Sigma$-type can refer to variables in its $fst$ and $snd$ types, and $fst$ itself in the type of $snd$. $\Sigma$-types are one of the canonical dependent types, alongside dependent products. There is a known equivalence between $\Pi$ and $\Sigma$ in that a function $\Pi(x : \alpha)(y : \beta x), \gamma x y$ can be transformed into a dependent function $\Pi(s : \Sigma(x : \alpha), \beta x), \gamma fst(s) snd(s)$, and vice-versa ("currying"-"uncurrying").

In the untyped categorical combinators, Uncurry can be written as:

$$\begin{aligned} Curry(x) &:= \Lambda(x) \\ Uncurry(x) &:= App \circ \langle x \circ Fst, Snd \rangle \\ Uncurry(x)(y, z) &= xyz \end{aligned} \tag{4}$$

Naively interpreting couples as $\Sigma$ pairs with point-free type arguments makes this example (**??**) quite difficult, especially in an affine setting. In $Uncurry(x)$, $snd\ (z : \beta y)$ depends on the $fst$ component $(y : \alpha)$ of the encapsulating couple's preimage, despite $y$ being destroyed upon reduction. $App$ has no knowledge of $y$, and thus cannot compute the type of $(xy, z)$. We remedy this by indexing $\Sigma$ with functions of the environment $f_\beta : \Pi(s : \delta)(x : f_\alpha s), \text{Sort } n$ (5) that determine the type of $snd$.

$$\frac{\textsc{Sigma}}{\alpha : \text{Sort } m, f_\beta : \delta \to \alpha \to \text{Sort } n}{\Sigma(f_\beta) : \delta \to \text{Sort } max(m, n)} \tag{5}$$

Figure 3: Where $\delta$ denotes an arbitrary environment and $\alpha$ denotes the $fst$ component. $\Sigma(f_\beta)(env : \delta) : \text{Sort } max(m, n)$

We thus define couples $[\![\beta]\!](a, b)$ as a degenerate case of $\Sigma$ where $\beta$ depends only on $fst$ and not the environment. We represent a no-op indexing of the environment with a unit type $\top$ so as to avoid universe level arguments to $\Sigma$. We use the notation $Coup(\beta)$ to refer to the type of a couple where $\beta : \alpha \to \text{Sort } n$ (**??**). We always refer to couples with their $\beta$ points, since they cannot be inferred.

$$\frac{}{\top : \text{Sort } 0}\ \text{Top} \qquad \frac{}{* : \top}\ \text{Unit} \qquad \frac{A : \text{Sort } m, B : A \to \text{Sort } n, x : A, y : Bx,}{[\![B]\!](x, y) : \Sigma(\text{`(Sort } m, B))A}\ \text{Coup} \qquad \frac{}{Coup(B) \coloneqq (\Sigma(\text{`}(\top, B))*)}$$

This view of couples as $\Sigma$-types naturally encodes a telescope context, and proves to be invaluable when compiling dependently-typed $\lambda$-calculi to the categorical combinators. A context $\Gamma$ can be encoded as a nested couple:

$$x_0 : A_0, x_1 : A_1(x_0), x_2 : A_2(x_0, x_1) \approx [\![\Sigma(A_1), A_2]\!]([\![A_1]\!](x_0, x_1), x_2) \tag{6}$$

We make use of this isomorphism to represent contexts in our ongoing effort to compile dependent $\lambda$-calculus to our combinators.

## 3.2 Type Arguments Index the Environment

All combinator objects have type annotations specified as points (usually as families indexing the environment). For example, the projection $Snd(f_\beta)$ is indexed by the term $f_\beta$ which depends on an arbitrary environment $(s : \delta)$ and the term $fst : \alpha$. Couple projection can be derived, and we refer to it as $Sndc(\beta)$.

## 3.3 Inference Rules

We make use of an infinite sort hierarchy to avoid paradoxes (i.e., Sort $m$ : Sort $m + 1$). Arrows are not first-class, and can be defined with $\Pi$. We do not explicitly apply the environment to the codomain in $\Pi$ in our written inference rules. The full types in our deep embedding are accessible in the appendix (A).

### 3.3.1 Application and $\Pi$

The inference rule for function application is as usual, except the codomain $\beta$ is applied the environment $s : \delta$, and the argument $x : f_\alpha s$. Since $f_\alpha s$ is discarded after type-checking, the environment can safely traverse the $\Pi$ object affinely (see inference rule (2)). We normalize types eagerly, but we do not always indicate so in our inference rules (see (A)). We use $\rightarrowtail^!_\eta$ to denote star-step affine reduction to a normal form composed with $\eta$-reduction to a normal form.

$$\frac{f : \Pi(f_\alpha, f_\beta)s, f_\alpha x \rightarrowtail^!_\eta \alpha \qquad x : \alpha_2, \alpha_2 \rightarrowtail^!_\eta \alpha \qquad f_\beta s x \rightarrowtail^!_\eta \beta}{fx : \beta}\ \text{App} \tag{7}$$

Figure 4: Inference rule: application

Since we do not make use of variables, $\Pi$ is itself a combinator dependent on an environment. Since the dependency is internalized by $f_\alpha$ and $f_\beta$, its type can be expressed with an arrow (2).

### 3.3.2 Arrow types

We define arrows $(A \to B)$ as $\Pi$ objects quoting their codomain to discard the environment-argument couple (8).

$$\frac{A : \text{Sort } m}{A \to B \coloneqq (\Pi(Id(\text{Sort } m), \Lambda(\text{Sort } m, Id(\text{Sort } m), \text{`}(Coup(Id(\text{Sort } m)), B)))A)} \tag{8}$$

### 3.3.3 Couple Projections

We define dependent and nondependent versions of the *Snd* projection to bootstrap several of the combinators' types. We also derive a *Sndc* combinator which accepts a $Coup(\beta)$ not depending on an environment. $Fst(\beta)$ depends on an environment, and $Fstc$ is similarly derived. Composition $(f \circ g)$

is nondependent with respect to $g$, although $f$ may depend on $gx$. A form of semi-dependency can be derived if $g$ carries its dependency in a couple $(gx : \Sigma(\ldots)\ldots)$.

We use thunking to reduce pairs ($\langle f, g\rangle x$) lazily. That is, $\langle f, g\rangle x$ is a normal form, and projections and evaluation are defined on it as if it were the reduced couple $(fx, gx)$. This approach enables some usage of the pair rule in our types, though we do not use it to define any of our core types.

The inference rules for projection on $\Sigma$-terms (couples and pairs) are as such:

$$\textsc{Fst}\ \frac{\alpha : \text{Sort } m, \delta : \text{Sort } o, f_\beta : \delta \to \alpha \to \text{Sort } n}{Fst(\beta) : \Pi(env : \delta)(s : \Sigma(f_\beta)env), \alpha} \qquad \textsc{Snd}\ \frac{\alpha : \text{Sort } m, \delta : \text{Sort } o, f_\beta : \delta \to \alpha \to \text{Sort } n}{Snd(\beta) : \Pi(env : \delta)(s : \Sigma(f_\beta)env), f_\beta env(Fst(f_\beta)(env)s))}$$

$$\textsc{Snd'}\ \frac{\alpha : \text{Sort } m, \beta : \text{Sort } n}{Snd'(\alpha, \beta) : Coup(`(\alpha, \beta)) \to \beta} \qquad \overline{Sndc(\beta) \coloneqq Snd(`(\top, \beta))*} \qquad \overline{Fstc(\beta) \coloneqq Fst(`(\top, \beta))*}$$

Our inference rules for projection apply to applied pairs as well, since they conceptually reduce to couples.

### 3.3.4 Identity

$Id$ accepts $\alpha$ as a point, but not a function of the environment. A type-point-free variant of $Id$ can be derived easily with Curry ($\Lambda$). The type of $Id(\alpha)$ is an arrow (9).

$$\textsc{Id}\ \frac{\alpha : \text{Sort } m}{Id(\alpha) : \alpha \to \alpha} \tag{9}$$

### 3.3.5 Composition

We have nondependent composition as in Curien's combinators, except that the codomain of $f$ can depend on $gx$, but the codomain of $g$ cannot depend on $x$ (10). Allowing both $f$ and $g$ to depend on their inputs would require duplicating the argument $x$ in our reduction rules, whereas $x$ is not usually duplicated in the categorical combinators.

$$\textsc{Comp.}\ \frac{f : \Pi(x : \beta), \gamma x, g : \beta \to \beta}{(f \circ g) : \Pi(x : \alpha), \gamma(gx)} \tag{10}$$

Figure 5: Inference rule: composition

Full dependency can be recovered when $g$ produces a $\Sigma$-term encoding its dependency (e.g., $g : \alpha \to Coup(\beta)$).

### 3.3.6 Pairs

Recall $\langle f, g\rangle x = (fx, gx)$. We do not define reduction directly on pairs, and instead give projections and evaluation rules for $\langle f, g\rangle x$ as if it were the reduced couple.

$$\textsc{Pair}\ \frac{f : \Pi(x : \alpha), \beta x \qquad g : \Pi(x : \alpha), \gamma x}{\langle f, g\rangle : \Pi(x : \alpha), \Sigma(\Lambda(\alpha, \beta, \gamma\ Fstc(\beta)))x} \tag{11}$$

### 3.3.7 Quotation

The quotation combinator ($`(\beta, x)$) is not dependent, but a dependent version similar to dependent $K$ can be derived.

$$\frac{\textsc{Quot}\quad \beta : \text{Sort } n, x : \alpha}{`(\beta, x) : \beta \to \alpha} \tag{12}$$

### 3.3.8 Curry

Recall $\Lambda(x)yz = x(y, z)$. Our inference rule for application (7) proves to be useful in defining the type, since terms in context can be used more than once in a $\Pi$ tree, so long as they are used at most once in each argument type.

$$\frac{\textsc{Curry}\quad \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n, x : (\Pi(s : Coup(\beta)), \gamma s)}{\Lambda(\alpha, \beta, x) : \Pi(x : \alpha)(y : \beta x), \Lambda(\alpha, \beta, \gamma)xy} \tag{13}$$

Figure 6: Inference rule: curry

### 3.3.9 App and Subst Combinators

As in Curien's combinators, the *App* combinator evaluates a couple as a computation. Its reduction rule is:

$$App(f, x) \to fx \tag{14}$$

Defining the type of *App* proves to be quite difficult in this configuration, since $[\![\beta]\!](f, x)$ is a $\Sigma$-couple. The term $x$ does not actually depend on $f$. The book-keeping required to discard $f$ in $\beta$ can be easily avoided by adopting the rule $App(x, f) \rightarrowtail (fx)$, as in Seely [8, p. 39].

We mirror the inference rule for $\Pi$ in that the types points for *App* index an environment. In fact, the types of the points $f_\alpha$ and $f_\beta$ are the exact same as in $\Pi$. Since *App* indexes the environment, we define its reduction rule as:

$$App(f_\alpha, f_\beta)env[\![\ldots]\!](x, f) \rightarrowtail fx \tag{15}$$

The inference rule requires duplicating the environment in the codomain of $App(\ldots, \ldots)env[\![\ldots]\!](x, f)$, since $f_\beta$ depends on the environment and the $Fst$ projection $x$, which also depends on the environment. However, the environment is not computationally relevant to the result of projection, and behaves more like a context. That is, duplicating the environment corresponds directly to context substitution. We can avoid the duplication altogether by defining *Subst* with respect to the combinator depending on the environment. The reduction and inference rule is defined for $Fst$ like such:

$$\begin{aligned} Subst(x, Fst(f_\beta))env[\![\ldots]\!](y, z) &\rightarrowtail xenvy \\ Subst(x, Fst(f_\beta))env[\![\ldots(\langle f, g\rangle z) &\rightarrowtail xenv(fz) \end{aligned} \tag{16}$$

$$\frac{\textsc{Subst Fst}\quad x : \Pi(env : \delta)(x : \alpha), \gamma envx}{Subst(x, Fst(f_\beta)) : \Pi(env : \delta)(s : \Sigma(f_\beta)), Subst(\gamma, Fst(f_|beta))} \tag{17}$$

We define the *App* inference rule (18) using our *Subst* combinator.

$$\frac{\textsc{Notation}\quad}{t_{\beta xy} \coloneqq \Lambda(\delta, f_\alpha, \Pi(f_\alpha, f_\beta) \circ Fstc(f_\alpha))}$$
$$\frac{\textsc{Appl}\quad \delta : \text{Sort } m, f_\alpha : \delta \to \text{Sort } m, f_\beta : \Pi(s : \delta)(x : f_\alpha s), \text{Sort } o}{App(f_\alpha, f_\beta) : \Pi(env : \delta)(s : \Sigma(t_{\beta xy})env), Subst(f_\beta, Fst(t_{\beta xy}))} \tag{18}$$

Figure 7: *App* inference rule

# 4 Affine and $\eta$-Reduction

In general, dependent combinatory logics require complex types. Hence, we extend Curien's $\eta$-reduction rules for the categorical combinators to enable some examples [4, p. 221]. Since our types are affine (3.3), we can realize decidable type-checking by demonstrating strong normalization of $\to_\eta^!$ and $\rightarrowtail^!$ and expressing conversion in our judgment relation as composition of $\rightarrowtail^!$ and $\to_\eta^!$ ($\rightarrowtail_\eta^!$).

We prove soundness and completeness of reduction functions for each scheme. We demonstrate strong normalization by a syntactic decreasing measure.

## 4.1 Automating Completeness

The derivations of completeness and soundness for affine and $\eta$-reduction are structurally quite similar. Thus, we make the practical design decision to abstract the notions of a size and step-like relations with Lean's typeclass system.

We identify a set of minimal requirements to prove completeness and strong normalization for our rules: soundness of a computable definition with respect to the inductive relation, a proof that a size measure decreases over the reduction, local confluence of the respective single-step relation, and "semi-completeness". We define these properties as typeclasses and instantiate them for $\to_\eta$ and $\rightarrowtail$. In doing so, we inherit global confluence by Nemann's lemma, a total, terminating reduction function, strong normalization, well-foundedness with respect to the relation, uniqueness of normal forms, completeness of the reduction function with respect to $\rightarrowtail^!$ and $\to_\eta^!$, and decidability of the reduction relation up to normal forms.

Our notion of a relation requires an inductive relation and a corresponding implementation with soundness:

```
class Rel.{u, v, s} {α : Type u} {β : Type s} (r : α → β → Type v)
  (m : Type (max v s) → Type (max v s) := Id) [hM : Monad m] where
  run (x : α) : m ((y : β) × (r x y))
  sound       : run x = pure ⟨y, h⟩ → r x y := fun _ => h
```

We use universe and monadic polymorphism to express single-step reduction with `Rel`, since single-step reduction is not always defined.

We define completeness with respect to a relation (`Rel`) as such:

```
open Rel in class Complete.{u, v, s} {α : Type u} {β : Type s} (r : α → β → Sort v)
  (P : β → Prop := fun _ => True)
  (m : Type (max v s) → Type (max v s) := Id) [hm : Monad m]
  [hR : Rel.{u, v, s} (fun (x : α) (y : β) => PULift.{v + 1, v} (r x y)) (m := m)] where
  complete : ∀ (h : r x y) (_p : P y), hR.run x = hm.pure ⟨y, PULift.up h⟩
```

We generalize the notion of a size measure, and automatically derive completeness with tactics:

```
open Rel in class Measure (r : Expr → ℕ → Prop)
  extends Rel.{0, 0, 0} (fun x n => PULift.{1, 0} (r x n)), Complete.{0} r where
  complete := by
    intro x y h _h
    induction h
    all_goals (
      simp_all [pure, id_pure_ext]
      try rfl
      try grind
    )

abbrev Measure.size (t : outParam (Expr → ℕ → Prop)) [hM : Measure t]
  (e : Expr) : ℕ := (hM.run e).fst
```

We define a reduction relation as a relation satisfying the decreasing measure property. We can usually derive the proof using tactics.

```
class RedRel (r : Expr → Expr → Prop) (t : outParam (Expr → ℕ → Prop))
  (m : Type → Type := Option) [_hM : Monad m]
  [hM : Measure t] extends Rel.{0, 0, 0} (fun (x : Expr)
  (y : Expr) => PULift.{1, 0} (r x y)) (m := m) where
  decreasing : r x y → (hM.run y).fst < (hM.run x).fst := by
    intro x y h
    induction h
    all_goals (
```

```
    try dsimp
    try simp_all [dcc_rel, dcc_size]
    try grind
    try aesop (rule_sets := [DccRel, DccSize])
    try rfl)
```

Relations with a decreasing measure inherit strong normalization, well-foundedness, and a total reduction function:

```
def RedRel.sn {r : Expr → Expr → Prop} (t : outParam (Expr → ℕ → Prop))
  {m : Type → Type} [Monad m] [hM : Measure t]
  (hR : RedRel r t (m := m) (hM := hM)) (e : Expr) : Acc (fun b a => r a b) e :=
    Acc.intro e (fun e' hr => by
    have h := hR.decreasing hr
    exact sn t hR e')
termination_by (hM.run e).fst

def RedRel.wellFounded {r : Expr → Expr → Prop} (t : outParam (Expr → ℕ → Prop))
  {m : Type → Type} [Monad m] [hM : Measure t]
  (hR : RedRel r t (m := m) (hM := hM)) : WellFounded (fun a b => r b a) := ⟨hR.sn⟩

def reduce {t : Expr → ℕ → Prop} [hT : Measure t] (r : Expr → Expr → Prop)
  [hR : RedRel r t (hM := hT)] (e : Expr) :
  ((e' : Expr) × (PULift.{1, 0} (Relation.ReflTransGen r e e'))) :=
  match hR.run e with
  | .some ⟨e', ⟨h⟩⟩ =>
    have _size_ : (hT.run e').fst < (hT.run e).fst := hR.decreasing h
    let ⟨e'', ⟨h₂⟩⟩ := reduce r (t := t) e'
    ⟨e'', ⟨.trans (.single h) h₂⟩⟩
  | .none => ⟨e, ⟨.refl⟩⟩
termination_by (hT.run e).fst

@[dcc_reduce, inline] abbrev reduce' {t : Expr → ℕ → Prop} [hT : Measure t]
  (r : Expr → Expr → Prop) [hR : RedRel r t (hM := hT)]
  (e : Expr) : Expr := (reduce r e).fst
```

Since $\rightarrowtail$ has congruence rules, confluence is required to show completeness of `reduce`. We define the notion like such:

```
open Rel in class Confluent.{u} {α : Type u} (r : α → α → Prop)
  (m : (α → α → Prop) → α → α → Prop := fun x => x) where
  conf : (m r A B) → (m r A C) → ∃ D, Relation.ReflTransGen.{u} r B D
  ∧ Relation.ReflTransGen.{u} r C D
```

We derive global confluence and uniqueness of normal forms for relations with a measure by Newmann's lemma.

**Theorem 1.** *If $r$ is locally confluent and strongly normalizing by a decreasing measure, then $r$ is globally confluent.*

*Proof.* By Newmann's lemma:

```
open Confluent in def RedRel.globally_confluent
  {r : Expr → Expr → Prop} {t : Expr → ℕ → Prop} {m : Type → Type}
  [hM : Measure t] [Monad m] [hC : Confluent r]
  [hR : RedRel r t (m := m) (hM := hM)] : (Confluent r (m := Relation.ReflTransGen)) where
  conf := fun {A B C} (h₁ : hR.ReflTrans A B) (h₂ : hR.ReflTrans A C) => by
    induction A using hR.wellFounded.induction generalizing B C with
    | h x ih =>
      match h₁.cases_head, h₂.cases_head with
      | .inl heq₁, .inl heq₂ =>
        exact ⟨x, heq₁ ▸ .refl, heq₂ ▸ .refl⟩
      | .inl heq₁, _ =>
        exact ⟨C, heq₁ ▸ h₂, .refl⟩
      | _, .inl heq₁ =>
        exact ⟨B, .refl, heq₁ ▸ h₁⟩
      | .inr ⟨C₁, hr₁, hrt₁⟩, .inr ⟨C₂, hr₂, hrt₂⟩ =>
        have ⟨D, hrt₁b, hrt₂b⟩ := hC.conf hr₁ hr₂
        have ⟨_, hrtt₁, hrtt₁b⟩ := ih _ hr₁ hrt₁ hrt₁b
        have ⟨_, hrtt₂, hrtt₂b⟩ := ih _ hr₂ hrt₂ hrt₂b
        have ⟨_, h₁, h₂⟩ := ih _ hr₁ (hrt₁b.trans hrtt₁b) (hrt₁b.trans hrtt₂b)
        exact ⟨_, .trans hrtt₁ h₁, .trans hrtt₂ h₂⟩
```

□

**Theorem 2.** *If $\to_r$ is confluent, and $A \to_r C$ and $A \to_r B$ and $B$ and $C$ are normal forms, then $B = C$.*
*Proof.*

```
def IsNorm.unique_refl {r : Expr → Expr → Prop} (hn : IsNorm r A)
  (hr : (Relation.ReflTransGen r) A B) : A = B := by
  rw [← Relation.reflGen_transGen] at hr
  cases hr with
  | refl => rfl
  | single h => exact hn.unique_trans h

open Confluent in @[dcc_rel, aesop safe destruct (rule_sets := [DccRel])]
  theorem norm_unique {r : Expr → Expr → Prop} {m : (Expr → Expr → Prop) → Expr → Expr → Prop}
  [hC : Confluent r m] (h₁ : m r A B) (hn₁ : IsNorm r B) (h₂ : m r A C) (hn₂ : IsNorm r C) : B
  ↪  = C := by
  have ⟨D, hr₁, hr₂⟩ := hC.conf h₁ h₂
  have ha := hn₁.unique_refl hr₁
  have hb := hn₂.unique_refl hr₂
  simp_all
```

□

We require a weak form of completeness to derive full completeness for `reduce`: if $A \to B$, then there exists some $C$ and a proof $h$ produced by `run` that $A \to C$:

```
open Rel in class SemiComplete {t : Expr → ℕ → Prop} [hM : Measure t]
  (r : Expr → Expr → Prop)
  [hR : RedRel r t (m := Option) (hM := hM)] where
  complete : r A B → ∃ B h, (hR.run A) = .some ⟨B, ⟨h⟩⟩
```

Relations with a measure, semi-completeness, and confluence inherit completeness of `reduce` with respect to normal forms.

**Theorem 3.** *Suppose $\to_r$ is strongly normalizing under a decreasing measure, semi-complete, and confluent. Then, if `hr` is a proof that $A \to_r^* B$, and $B$ is a normal form, then `reduce A = (B, hr)`.*

*Proof.* By case analysis on the definition of `reduce`.

```
theorem reduce_complete {r : Expr → Expr → Prop} {t : Expr → ℕ → Prop}
  [hM : Measure t] [hR : RedRel r t (m := Option.{0}) (hM := hM)] [hN : SemiComplete r (hR :=
  ↪  hR)] [Confluent r]
  {A B : Expr} (hr : Relation.ReflTransGen r A B) (hn : IsNorm r B) : (reduce r A) = ⟨B, ⟨hr⟩⟩
  ↪  := by
  simp [reduce]
  split
  case h_1 B' hr₂ heq =>
    have ⟨C, h₁, h₂⟩ := hR.globally_confluent.conf hr (.single hr₂)
    have h_b_c_eq := hn.unique_refl h₁
    have hr_b'_b := h_b_c_eq ▸ h₂
    match hr_b'_b.cases_head with
    | .inl h =>
      have h_red_norm := hR.reduce_norm_fst hn
      cases h_b_c_eq; cases h
      congr
      repeat simp_all
    | .inr ⟨_, hrB', hrt⟩ =>
      rename_i w
      have h_dec : (hM.size t B') < (hM.size t A) := by
        have h₁ := hR.decreasing hr₂
        have h₂ := hR.decreasing hrB'
        grind
      have h := reduce_complete (.trans (.single hrB') hrt) hn
      have h_fst : (reduce r B').fst = B := by
        simp_all
      cases h_fst
      congr
  case h_2 hneq =>
    match hr.cases_head with
    | .inl h => cases h; congr
    | .inr ⟨_, hrb', hrt⟩ =>
      have ⟨_, hA₁, hA₂⟩ := hN.complete hrb'
      simp_all
termination_by ((hM.size t A), (hM.size t B))
```

□

## 4.2 Affine Reduction Rules

Our affine reduction rules are mostly the same as in the categorical abstract machine [3], except we omit the general pair rule, define a context substitution combinator *Subst* (16), and flip the order of the couple argument to *App*:

```
inductive RedAff : Expr → Expr → Prop
  | subst₁p  : RedAff (DCC (Subst(x, Fst(f_β)) env (⟨f, g⟩ z))) (DCC (x env (f z)))
  | subst₁   : RedAff (DCC (Subst(x, Fst(f_β)) env ⟦β⟧(y, z))) (DCC (x env y))
  | subst₂   : RedAff (DCC (Subst(x, Snd(f_β)) env ⟦β⟧(y, z))) (DCC (x env z))
  | snd'     : RedAff (DCC (Snd'(_δ, _τ) ⟦β⟧(a, b))) b
  | quot     : RedAff (DCC (`(β, x) y)) x
  | id       : RedAff (DCC (Id(α) x)) x
  | appl     : RedAff (DCC (App(_α, _β) env ⟦β⟧(x, y))) (DCC (x y))
  | fst      : RedAff (DCC (Fst(_α) _env ⟦β⟧(a, b))) a
  | fstp     : RedAff (DCC (Fst(_α) _env (⟨f, g⟩ x))) (DCC (f x))
  | snd      : RedAff (DCC (Snd(_α) _env ⟦β⟧(a, b))) b
  | sndp     : RedAff (DCC (Snd(_α) _env (⟨f, g⟩ x))) (DCC (g x))
  | comp     : RedAff (DCC ((x ∘ y) z)) (DCC (x (y z)))
  | curr     : RedAff (DCC (Λ(α, β, x) y z)) (DCC (x ⟦β⟧(y, z)))
  | sigl     : RedAff x x' → RedAff (DCC ⟦β⟧(x, y)) (DCC ⟦β⟧(x', y))
  | sigr     : RedAff y y' → RedAff (DCC ⟦β⟧(x, y)) (DCC ⟦β⟧(x, y'))
  | congrFun : RedAff f f' → RedAff (DCC (f x)) (DCC (f' x))
  | congrArg : RedAff x x' → RedAff (DCC (f x)) (DCC (f x'))
```

We compute single-step affine reduction like such:

```
def step? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (RedAff e e'))) :=
  match e with
  | (DCC (Subst(_x, Fst(_f_β)) _env (⟨_f, _g⟩ _z))) => .some ⟨_, ⟨.subst₁p⟩⟩
  | (DCC (Subst(_x, Fst(_f_β)) _env ⟦_β⟧(_y, _z))) => .some ⟨_, ⟨.subst₁⟩⟩
  | (DCC (Subst(_x, Snd(_f_β)) _env ⟦_β⟧(_y, _z))) => .some ⟨_, ⟨.subst₂⟩⟩
  | (DCC (Snd'(_α, _τ) ⟦_β⟧(_a, _b)))              => .some ⟨_, ⟨.snd'⟩⟩
  | (DCC (`(_β, _x) _y))                           => .some ⟨_, ⟨.quot⟩⟩
  | (DCC (Id(_α) _x))                              => .some ⟨_, ⟨.id⟩⟩
  | (DCC (App(_α, _β) _env ⟦_β₂⟧(_x, _y)))         => .some ⟨_, ⟨.appl⟩⟩
  | (DCC (Fst(_α) _env ⟦_γ⟧(_a, _b)))              => .some ⟨_, ⟨.fst⟩⟩
  | (DCC (Fst(_α) _env (⟨_f, _g⟩ _x)))             => .some ⟨_, ⟨.fstp⟩⟩
  | (DCC (Snd(_α) _env ⟦_γ⟧(_a, _b)))              => .some ⟨_, ⟨.snd⟩⟩
  | (DCC (Snd(_α) _env (⟨_f, _g⟩ _x)))             => .some ⟨_, ⟨.sndp⟩⟩
  | (DCC ((x ∘ y) z))                              => .some ⟨(DCC (x (y z))), ⟨.comp⟩⟩
  | (DCC (Λ(_α, β, x) y z))                        => .some ⟨(DCC (x ⟦β⟧(y, z))), ⟨.curr⟩⟩
  | DCC ⟦β⟧(x, y) =>
    match x.step?, y.step? with
    | .some ⟨x', ⟨h₁⟩⟩, _ => .some ⟨(DCC ⟦β⟧(x', y)), ⟨.sigl h₁⟩⟩
    | _, .some ⟨y', ⟨h₁⟩⟩ => .some ⟨(DCC ⟦β⟧(x, y')), ⟨.sigr h₁⟩⟩
    | _, _ => .none
  | DCC (f x) =>
    match f.step?, x.step? with
    | .some ⟨f', ⟨h₁⟩⟩, _ => .some ⟨(DCC (f' x)), ⟨.congrFun h₁⟩⟩
    | _, .some ⟨x', ⟨h₁⟩⟩ => .some ⟨(DCC (f x')), ⟨.congrArg h₁⟩⟩
    | _, _ => .none
  | _ => .none
```

Note that none of our $\rightarrowtail$ rules duplicate their term arguments. Furthermore, each rule (except for congruence) eliminates a combinator. We thus define a syntactic notion of term size:

```
inductive Size : Expr → ℕ → Prop
  | tintro : Size (DCC *) 0
  | top    : Size (DCC ⊤) 0
  | bot    : Size (DCC ⊥) 0
  | subst  : Size x m → Size y n → Size (DCC Subst(x, y)) (m + n).succ
  | sig    : Size (DCC Σ(β)) 0
  | coup   : Size x m → Size y n → Size (DCC ⟦β⟧(x, y)) (m + n)
  | sort   : Size (DCC Sort m) 0
  | pi     : Size (DCC Π(α, β)) 0
  | quot   : Size x m → Size (DCC `(β, x)) m.succ
  | id     : Size (DCC Id(α)) 1
  | fst    : Size (DCC Fst(_α)) 1
  | snd'   : Size (DCC Snd'(δ, β)) 1
  | snd    : Size (DCC Snd(_α)) 1
  | appl   : Size (DCC App(_α, β)) 1
  | comp   : Size f m → Size g n → Size (DCC (f ∘ g)) (m + n).succ
  | pair   : Size f m → Size g n → Size (DCC ⟨f, g⟩) (m + n).succ
```

```
  | curr   : Size x m → Size (DCC Λ(α, β, x)) m.succ
  | app    : Size f m → Size x n → Size (DCC (f x)) (m + n)
```

Except for objects with no reduction rules, sizes are given by the successor of the sum of the constituent term point sizes. For example, type points like $\alpha$ in $Snd'$ have no weight, since they are computationally irrelevant.

We compute term size like such:

```
@[aesop norm (rule_sets := [DccSize]), dcc_size, grind]
def size (e : Expr) : ((n : ℕ) × (PULift.{1, 0} (Size e n))) :=
  match e with
  | DCC ⋆ => ⟨_, ⟨.tintro⟩⟩
  | DCC ⊤ => ⟨_, ⟨.top⟩⟩
  | DCC ⊥ => ⟨_, ⟨.bot⟩⟩
  | DCC Subst(x, y) =>
    let ⟨_, ⟨h₁⟩⟩ := x.size
    let ⟨_, ⟨h₂⟩⟩ := y.size
    ⟨_, ⟨.subst h₁ h₂⟩⟩
  | DCC Sort _m => ⟨_, ⟨.sort⟩⟩
  | DCC Σ(_β) => ⟨_, ⟨.sig⟩⟩
  | DCC `(_β, x) =>
    let ⟨_, ⟨h₁⟩⟩ := x.size
    ⟨_, ⟨.quot h₁⟩⟩
  | DCC ⟦_β⟧(x, y) =>
    let ⟨_, ⟨h₁⟩⟩ := x.size
    let ⟨_, ⟨h₂⟩⟩ := y.size
    ⟨_, ⟨.coup h₁ h₂⟩⟩
  | DCC Id(_α)       => ⟨_, ⟨.id⟩⟩
  | DCC Fst(_α) => ⟨_, ⟨.fst⟩⟩
  | DCC Snd'(_α,_β) => ⟨_, ⟨.snd'⟩⟩
  | DCC Snd(_α) => ⟨_, ⟨.snd⟩⟩
  | DCC Π(_α, _β) => ⟨_, ⟨.pi⟩⟩
  | DCC App(_α, _β) => ⟨_, ⟨.appl⟩⟩
  | DCC (f ∘ g) =>
    let ⟨_m, ⟨h₁⟩⟩ := f.size
    let ⟨_n, ⟨h₂⟩⟩ := g.size
    ⟨_, ⟨.comp h₁ h₂⟩⟩
  | DCC ⟨a, b⟩ =>
    let ⟨_o, ⟨h₁⟩⟩ := a.size
    let ⟨_p, ⟨h₂⟩⟩ := b.size
    ⟨_, ⟨.pair h₁ h₂⟩⟩
  | DCC Λ(_α, _β, x) =>
    let ⟨_n, ⟨h₁⟩⟩ := x.size
    ⟨_, ⟨.curr h₁⟩⟩
  | DCC (f x) =>
    let ⟨_, ⟨h₁⟩⟩ := f.size
    let ⟨_, ⟨h₂⟩⟩ := x.size
    ⟨_, ⟨.app h₁ h₂⟩⟩

@[dcc_rel, aesop norm (rule_sets := [DccRel])]
instance Size.instMeasure : Measure Size where
  run := size
```

## 4.3 Strong Normalization of Affine Reduction

Using our size measure, typeclasses, and tactic automation, we show strong normalization.

**Theorem 4.** *If $e$ is an expression, then $e$ is strongly-normalizing under $\rightarrowtail$.*

*Proof.*

```
instance RedAff.instRedRel : RedRel RedAff Size where
  run := step?

theorem red_aff_sn : ∀ (e : Expr), Acc (fun b a => RedAff a b) e :=
  RedAff.instRedRel.sn
```

□

**Theorem 5.** *If $e$ is an expression, then $e$ is strongly-normalizing under $\rightarrowtail^+$.*

*Proof.*

```
theorem red_aff_trans_sn : ∀ (e : Expr), Acc
  (fun b a => Relation.TransGen RedAff a b) e :=
  RedAff.instRedRel.to_trans.sn
```

□

## 4.4 Confluence of Affine Reduction

We demonstrate local confluence by induction on the reduction relation.

**Theorem 6.** *If $A \rightarrowtail B$ and $A \rightarrowtail C$, then there exists $D$, such that $B \rightarrowtail^* D$ and $C \rightarrowtail^* D$.*

*Proof.* We proceed by induction on the reduction relation. We have omitted several repetitive cases for brevity (see (A)).

```
open Congruent in @[dcc_reduce, aesop safe apply (rule_sets := [DccReduce])] theorem
↪  RedAff.conf (h₁ : RedAff A B) (h₂ : RedAff A C) : ∃ D, RedAff.instRedRel.ReflTrans B D ∧
↪  RedAff.instRedRel.ReflTrans C D := by
  induction h₁ generalizing C with
  | subst₁p =>
    cases h₂ with
    | subst₁p => exact ⟨_, .refl, .refl⟩
    | congrFun h => cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, congrFun' (congrArg' (.single h)), .single .subst₁p⟩
    | congrArg h => cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, congrArg' (congrArg' (.single h)), .single .subst₁p⟩
  | subst₁ =>
    cases h₂ with
    | subst₁ => exact ⟨_, .refl, .refl⟩
    | congrFun h => cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, congrFun' (congrArg' (.single h)), .single .subst₁⟩
    | congrArg h => cases h with
      | sigl h => exact ⟨_, congrArg' (.single h), .single .subst₁⟩
      | sigr h => exact ⟨_, .refl, .single .subst₁⟩
  | appl =>
    cases h₂ with
    | appl => exact ⟨_, .refl, .refl⟩
    | congrFun h => cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .refl, .single .appl⟩
    | congrArg h =>
      cases h with
      | sigl h => exact ⟨_, .single (.congrFun h), .single .appl⟩
      | sigr h => exact ⟨_, .single (.congrArg h), .single .appl⟩
  | congrFun h_r ih =>
    cases h₂
    case subst₁p =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .subst₁p, congrFun' (congrArg' (.single h))⟩
    case snd' | quot | id | comp => cases h_r
    case congrFun f f' x f'' h =>
      have ⟨_, h₁, h₂⟩ := ih h
      exact ⟨_, congrFun' h₁, congrFun' h₂⟩
    case congrArg f f' x x'' h_x =>
      exact ⟨_, congrArg' (.single h_x), congrFun' (.single h_r)⟩
    case _ => sorry -- omitted for brevity
  | _ => sorry -- omitted for brevity

@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance RedAff.instConf : Confluent RedAff :=
↪  ⟨RedAff.conf⟩
```

□

**Theorem 7.** *If $A \rightarrowtail^* B$ and $A \rightarrowtail^* C$, then there exists $D$, such that $B \rightarrowtail^* D$ and $C \rightarrowtail^* D$.*

*Proof.* By Newmann's lemma.

```
theorem red_aff_globally_conf : Relation.ReflTransGen RedAff A B
  → Relation.ReflTransGen RedAff A C
  → ∃ D, Relation.ReflTransGen RedAff B D
  ∧ Relation.ReflTransGen RedAff C D :=
  RedAff.instRedRel.globally_confluent.conf
```

□

## 4.5 Completeness of an affine reducer

Finally, we show semi-completeness of `step?` (4.1) and completeness of `reduce` with respect to normal forms under $\rightarrowtail^*$.

**Theorem 8.** *If $A \rightarrowtail B$, then there exists $B'$ and a proof $h$ that $A \rightarrowtail B'$ such that step?$A$ = some$(B', h)$.*

*Proof.*

```
def semi_complete {A B : Expr} (hr : A.RedAff B) : ∃ B', ∃ (h : A.RedAff B'), (step? A) =
↪  .some ⟨B', ⟨h⟩⟩ := by
  change ∃ _, ∃ _, (step? _) = .some _
  unfold step?
  split
  repeat exact ⟨_, (RedAff.instRedRel.sound rfl).down, rfl⟩
  case h_14 =>
    cases hr with
    | sigl h_x =>
      have ⟨_, hrx', heq'⟩ := semi_complete h_x
      exact ⟨_, .sigl (by assumption), by rw [heq']⟩
    | sigr h_y =>
      rename_i x y _
      have ⟨_, hry', heq'⟩ := semi_complete h_y
      match x.step? with
      | .none =>
        exact ⟨_, .sigr (by assumption), by rw [heq']⟩
      | .some ⟨_, ⟨h_x'⟩⟩ =>
        exact ⟨_, .sigl (by assumption), by rw [heq']⟩
  case h_15 => sorry -- omitted for brevity
  case h_16 => sorry -- omitted for brevity

@[dcc_rel, aesop norm (rule_sets := [DccRel])]
instance RedAff.instSemicomplete : SemiComplete RedAff := ⟨semi_complete⟩
```

□

**Theorem 9.** *If $h$ is a proof that $A \rightarrowtail^* B$ and $B$ is a normal form, then reduce$A$ = $(B, h)$.*

*Proof.*

```
theorem aff_complete (h : Relation.ReflTransGen RedAff A B)
  (hn : IsNorm RedAff B) : reduce RedAff A = ⟨B, ⟨h⟩⟩ :=
  RedAff.instRedRel.instComplete.complete h hn
```

□

## 4.6 $\eta$-Reduction

To simplify type expressions, we employ Curien's $\eta$-reduction rules [4, p. 221], and extend them to $\Pi$ and $\Sigma$. We do not have congruence, and as such inherit deterministic reduction. We define $\rightarrow_\eta$ like such:

```
inductive RedEta : Expr → Expr → Prop
  | curr      : RedEta x x' → RedEta (DCC (Λ(α, β, x))) (DCC (Λ(α, β, x')))
  | curr_quot : RedEta (DCC (Λ(α, Id(α), `(Coup(Id(α)), x)) y)) (DCC `(y, x))
  | curr_fst  : RedEta (DCC (Λ(α, β, x ∘ Fstc(β₂)) y)) (DCC `(β, x))
  | pi        : RedEta (DCC Π(α, β)) p' → RedEta (DCC (Π(α, β) env)) (DCC (p' env))
  | pi_quot   : RedEta x x' → RedEta (DCC (Π(α, `(β, x)))) (DCC (Π(α, `(β, x'))))
  | pi'       : RedEta β β' → RedEta (DCC (Π(α, β))) (DCC (Π(α, β')))
  | sig       : RedEta (DCC (Σ(β))) b' → RedEta (DCC (Σ(β) env)) (DCC (b' env))
  | sig'      : RedEta β β' → RedEta (DCC (Σ(β))) (DCC Σ(β'))
  | idl       : RedEta (DCC (Id(α) ∘ f)) f
  | fst       : RedEta (DCC (Fst(_β) _env) ∘ ⟨x, y⟩) x
  | snd       : RedEta (DCC (Snd(_β) _env) ∘ ⟨x, y⟩) y
  | beta      : RedEta
    (DCC ((App(_f_α, _f_β) _env) ∘ ⟨Λ(α, _β, x), y⟩))
    (DCC (x ∘ ⟨Id(α), y⟩))
```

```
  | ai         : RedEta (DCC Λ(α, β, (App(f_α, f_β) env))) (DCC Id((Π(f_α, f_β) env)))
  | fsi        : RedEta (DCC ⟨(Fst(_β) env), (Snd(_β₂) env₂)⟩) (DCC Id((Σ(_β) env)))
  | uncur      : RedEta (DCC Λ(α, β,
    (App(_f_α, _f_β) _env) ∘ ⟨f ∘ (Fst(_x) _env₁), (Snd(_x₂) _env₂)⟩)) f
```

We only $\eta$-reduce $\Pi$ types with respect to their codomains, since their domains generally do not perform significant modification to the environment, since it has already been processed externally. We also omit Curien's *Idr* rule, since it does not occur frequently in our types.

We compute $\to_\eta$ like such:

```
@[dcc_step, aesop norm (rule_sets := [DccStep])]
def step_eta? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (RedEta e e'))) :=
  match e with
  | DCC (Λ(α, _b, `(_c, x)) y) =>
    if heq : DCC (Λ(α, _b, `(_c, x)) y) = (DCC (Λ(α, Id(α), `(Coup(Id(α)), x)) y)) then
      heq ▸ .some ⟨_, ⟨.curr_quot⟩⟩
    else .none
  | DCC (Λ(_α, _β, _x ∘ Fstc(_β₂)) _y) => .some ⟨_, ⟨.curr_fst⟩⟩
  | DCC Σ(f_β) =>
    match f_β.step_eta? with
    | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.sig' h⟩⟩
    | _ => .none
  | DCC (Σ(f_β) _env) =>
    match (DCC Σ(f_β)).step_eta? with
    | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.sig h⟩⟩
    | _ => .none
  | DCC (Π(_f_α, f_β) _env) =>
    match (DCC Π(_f_α, f_β)).step_eta? with
    | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.pi h⟩⟩
    | _ => .none
  | DCC (Π(_f_α, f_β)) =>
    match f_β with
    | DCC `(_β, x) =>
      match x.step_eta? with
      | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.pi_quot h⟩⟩
      | _ => .none
    | x =>
      match x.step_eta? with
      | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.pi' h⟩⟩
      | _ => .none
  | (DCC (Id(_α) ∘ _f)) => .some ⟨_, ⟨.idl⟩⟩
  | (DCC (Fst(_β) _env) ∘ ⟨_x, _y⟩) => .some ⟨_, ⟨.fst⟩⟩
  | (DCC (Snd(_β) _env) ∘ ⟨_x, _y⟩) => .some ⟨_, ⟨.snd⟩⟩
  | DCC Λ(α, β, x) =>
    match x.step_eta? with
    | .some ⟨_, ⟨h⟩⟩ => .some ⟨_, ⟨.curr h⟩⟩
    | .none =>
      match DCC Λ(α, β, x) with
      | (DCC Λ(_α, _β, (App(_f_α, _f_β) _env))) => .some ⟨_, ⟨.ai⟩⟩
      | (DCC Λ(_α, _β,
        (App(_f_α, _f_β) _env) ∘ ⟨_f ∘ (Fst(_x) _env₁), (Snd(_x₂) _env₂)⟩)) =>
        .some ⟨_, ⟨.uncur⟩⟩
      | _ => .none
  | (DCC ((App(_f_α, _f_β) _env) ∘ ⟨Λ(__α, __β, _x), _y⟩)) => .some ⟨_, ⟨.beta⟩⟩
  | (DCC ⟨(Fst(_β) _env), (Snd(_β₂) _env₂)⟩) => .some ⟨_, ⟨.fsi⟩⟩
  | _ => .none
```

### 4.6.1 Syntactic size under $\eta$-reduction

We define a size measure for $\to_\eta$, since $\eta$-reduction modifies $\Pi$ codomains, whereas affine reduction does not. The size measure is exactly the same, except type terms are given weights:

```
inductive EtaSize : Expr → ℕ → Prop
  | tintro : EtaSize (DCC *) 1
  | top    : EtaSize (DCC ⊤) 1
  | bot    : EtaSize (DCC ⊥) 1
  | subst  : EtaSize x m → EtaSize y n → EtaSize (DCC Subst(x, y)) (m + n).succ.succ
  | sig    : EtaSize β m → EtaSize (DCC Σ(β)) m.succ
  | coup   : EtaSize x m → EtaSize y n → EtaSize (DCC ⟦β⟧(x, y)) (m + n).succ
  | sort   : EtaSize (DCC Sort m) 1
  | pi     : EtaSize β m → EtaSize (DCC Π(α, β)) m.succ
  | quot   : EtaSize x m → EtaSize (DCC `(β, x)) m.succ.succ
  | id     : EtaSize (DCC Id(α)) 2
```

```
  | fst    : EtaSize (DCC Fst(_α)) 2
  | snd'   : EtaSize (DCC Snd'(δ, β)) 2
  | snd    : EtaSize (DCC Snd(_α)) 2
  | appl   : EtaSize (DCC App(_α, β)) 2
  | comp   : EtaSize f m → EtaSize g n → EtaSize (DCC (f ∘ g)) (m + n).succ.succ
  | pair   : EtaSize f m → EtaSize g n → EtaSize (DCC ⟨f, g⟩) (m + n).succ.succ
  | curr   : EtaSize x m → EtaSize (DCC Λ(α, β, x)) m.succ.succ
  | app    : EtaSize f m → EtaSize x n → EtaSize (DCC (f x)) (m + n)
```

We compute $\eta$-term size like such:

```
@[aesop norm (rule_sets := [DccSize]), dcc_size, grind]
def eta_size (e : Expr) : ((n : ℕ) × (PULift.{1, 0} (EtaSize e n))) :=
  match e with
  | DCC ⋆ => ⟨_, ⟨.tintro⟩⟩
  | DCC ⊤ => ⟨_, ⟨.top⟩⟩
  | DCC ⊥ => ⟨_, ⟨.bot⟩⟩
  | DCC Subst(x, y) =>
    let ⟨_, ⟨h₁⟩⟩ := x.eta_size
    let ⟨_, ⟨h₂⟩⟩ := y.eta_size
    ⟨_, ⟨.subst h₁ h₂⟩⟩
  | DCC Sort _m => ⟨_, ⟨.sort⟩⟩
  | DCC Σ(β) =>
    let ⟨_, ⟨h₁⟩⟩ := β.eta_size
    ⟨_, ⟨.sig h₁⟩⟩
  | DCC `(_β, x) =>
    let ⟨_, ⟨h₁⟩⟩ := x.eta_size
    ⟨_, ⟨.quot h₁⟩⟩
  | DCC ⟦_β⟧(x, y) =>
    let ⟨_, ⟨h₁⟩⟩ := x.eta_size
    let ⟨_, ⟨h₂⟩⟩ := y.eta_size
    ⟨_, ⟨.coup h₁ h₂⟩⟩
  | DCC Id(_α)       => ⟨_, ⟨.id⟩⟩
  | DCC Fst(_α) => ⟨_, ⟨.fst⟩⟩
  | DCC Snd'(_α,_β) => ⟨_, ⟨.snd'⟩⟩
  | DCC Snd(_α) => ⟨_, ⟨.snd⟩⟩
  | DCC Π(_α, β) =>
    let ⟨_, ⟨h₁⟩⟩ := β.eta_size
    ⟨_, ⟨.pi h₁⟩⟩
  | DCC App(_α, _β)  => ⟨_, ⟨.appl⟩⟩
  | DCC (f ∘ g) =>
    let ⟨_m, ⟨h₁⟩⟩ := f.eta_size
    let ⟨_n, ⟨h₂⟩⟩ := g.eta_size
    ⟨_, ⟨.comp h₁ h₂⟩⟩
  | DCC ⟨a, b⟩ =>
    let ⟨_o, ⟨h₁⟩⟩ := a.eta_size
    let ⟨_p, ⟨h₂⟩⟩ := b.eta_size
    ⟨_, ⟨.pair h₁ h₂⟩⟩
  | DCC Λ(_α, _β, x) =>
    let ⟨_n, ⟨h₁⟩⟩ := x.eta_size
    ⟨_, ⟨.curr h₁⟩⟩
  | DCC (f x) =>
    let ⟨_, ⟨h₁⟩⟩ := f.eta_size
    let ⟨_, ⟨h₂⟩⟩ := x.eta_size
    ⟨_, ⟨.app h₁ h₂⟩⟩

@[dcc_rel, aesop norm (rule_sets := [DccRel])]
instance EtaSize.instMeasure : Measure EtaSize where
  run := eta_size
```

### 4.6.2 $\eta$ Completeness

We derive strong normalization from a `RedRel` instance.

**Theorem 10.** *If $e$ is an expression, then $e$ is strongly normalizing under $\to_\eta$.*

*Proof.*

```
@[dcc_rel, aesop norm (rule_sets := [DccRel])]
instance RedEta.instRedRel : RedRel RedEta EtaSize where
  run := step_eta?

theorem red_eta_sn : ∀ (e : Expr), Acc (fun b a => RedEta a b) e := RedEta.instRedRel.sn
```

□

We prove confluence by demonstrating that single-step $\eta$-reduction is deterministic.

**Theorem 11.** *If $A \to_\eta B$ and $A \to_\eta C$, then $B = C$.*

*Proof.* We proceed by induction on the reduction relation.

```
@[simp] theorem RedEta.unique (h₁ : RedEta A B) (h₂ : RedEta A C) : B = C := by
  induction h₁ generalizing C with
  | sig' | pi | curr_fst | sig => cases h₂; aesop
  | pi_quot hr ih =>
    cases h₂
    · rename_i hr₂; aesop
    · rename_i hr₂; cases hr₂
  | pi' hr ih =>
    cases h₂
    · rename_i hr₂; cases hr
    · aesop
  | curr hr ih =>
    cases h₂ with
    | curr hr₂ => rw [ih hr₂]
    | ai | uncur => cases hr
  | ai | uncur =>
    cases h₂
    · rename_i h; cases h
    · rfl
  | curr_quot | snd | fst | idl | beta | fsi => cases h₂; rfl
```

□

**Theorem 12.** *If $A \to_\eta B$ and $A \to_\eta C$, then there exists $D$, such that $B \to_\eta^* D$ and $C \to_\eta^* D$.*

*Proof.* (4.6.2) By uniqueness under $\to_\eta$:

```
theorem RedEta.conf (h₁ : RedEta A B) (h₂ : RedEta A C) :
  ∃ D, Relation.ReflTransGen B D
  ∧ Relation.ReflTransGen C D := by
  have h := h₁.unique h₂
  exact ⟨B, h ▸ .refl, h ▸ .refl⟩
```

□

**Theorem 13.** *If $A \to_\eta^* B$ and $A \to_\eta^* C$, then there exists $D$, such that $B \to_\eta^* D$ and $C \to_\eta^* D$.*

*Proof.* By Newmann's lemma.

```
@[dcc_rel, aesop norm (rule_sets := [DccRel])]
instance RedEta.instConf : Confluent RedEta := ⟨RedEta.conf⟩

theorem red_eta_globally_conf : Relation.ReflTransGen RedEta A B → Relation.ReflTransGen
↪  RedEta A C → ∃ D,
  Relation.ReflTransGen RedEta B D ∧ Relation.ReflTransGen RedEta C D :=
  RedEta.instRedRel.globally_confluent.conf
```

□

Due to determinism, we have completeness under single-step $\to_\eta$.

**Theorem 14.** *If $h$ is a proof that $A \to_\eta B$, then $step_eta?A = some(B, h)$.*

*Proof.* We proceed by induction on the step relation.

```
theorem RedEta.complete (h_red : RedEta A B) : step_eta? A = .some ⟨B, ⟨h_red⟩⟩ := by
  induction h_red <;> (
    unfold step_eta?
    aesop (rule_sets := [DccStep, DccReduce, DccSize, DccRel]))
```

□

As in $\rightarrowtail$, we inherit completeness of full reduction with respect to $\to_\eta^*$ and normal forms.

**Theorem 15.** *If $h$ is a proof that $A \to_\eta^* B$ and $B$ is a normal form, then $reduceA = (B, h)$.*

*Proof.* By typeclass automation:

```
def eta_semi_complete {A B : Expr} (h_red : A.RedEta B) :
  ∃ B', ∃ (h : A.RedEta B'), (step_eta? A) = .some ⟨B', ⟨h⟩⟩ :=
  ⟨_, _, h_red.complete⟩

@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance RedEta.instSemicomplete : SemiComplete
↪  RedEta := ⟨eta_semi_complete⟩

theorem eta_complete (h : Relation.ReflTransGen RedEta A B)
  (hn : IsNorm RedEta B) : reduce RedEta A = ⟨B, ⟨h⟩⟩ :=
  RedEta.instRedRel.instComplete.complete h hn
```

□

# 5 Type Inference

Note that the types of all of the dependent categorical combinators are affine (3.3). We use our affine and $\eta$ reduction functions with soundness and completeness with respect to normal forms to prove decidability of type-checking.

We do not assume a general conversion constructor in our judgment relation. Instead, we eagerly $\beta$ and $\eta$ normalize our types. We do not refer to our computable reduction functions in the judgment relation. Instead, we define a predicate `Reduces` with completeness and soundness.

```
inductive Reduces : Expr → Expr → Prop
  | mk : Relation.ReflTransGen RedAff A B
    → IsNorm RedAff B
    → Relation.ReflTransGen RedEta B C
    → IsNorm RedEta C
    → Reduces A C

def reduce_all := reduce' RedEta ∘ reduce' RedAff

@[dcc_reduce, aesop safe forward (rule_sets := [DccReduce])]
theorem Reduces.unique : Reduces A B → Reduces A C → B = C := fun ⟨a, b, c, d⟩ ⟨g, h, i, j⟩ =>
↪  by
  have ha := norm_unique a b g h
  exact norm_unique c d (ha ▸ i) j

@[dcc_reduce, aesop safe destruct (rule_sets := [DccReduce])]
theorem Reduces.complete : Reduces A B → reduce_all A = B :=
  fun ⟨ha, hb, hc, hd⟩ => by
    unfold reduce_all
    unfold reduce'
    cases reduce_complete_fst ha hb; cases reduce_complete_fst hc hd; rfl

def reduce_reduces (h : reduce_all A = B) : Reduces A B := ⟨
  reduce_sound rfl,
  reduce_norm rfl,
  h ▸ (reduce_sound rfl),
  h ▸ (reduce_norm rfl)⟩
```

For example, application is inferred like such:

```
inductive Judge : Expr → Expr → Prop where
  | app    : Judge f (DCC (Π(f_α, f_β) x_α))
    → Reduces (DCC (f_α x_α)) α
    → Judge x α
    → Reduces (DCC (f_β x_α x)) τ
    → Judge (DCC (f x)) τ
```

For convenience, we define a function to pattern-match arrows, which are Π-objects defined as such:

```
@[dcc_judge] abbrev arrow (α β : Expr) (m : Level) : Expr :=
  DCC (Π(Id(Sort m), Λ(Sort m, Id(Sort m), `(Coup(Id(Sort m)), β))) α)

macro_rules
  | `(DCC ($α:expr : Sort $m) → $β:expr) => `(arrow (DCC $α) (DCC $β) $m)

@[dcc_judge, inline]
abbrev arrow.matcher (t : Expr) : Option ((α β : Expr) × (m : Level) × PULift.{1, 0} (t = (DCC
↪  (α : Sort m) → β))) :=
```

```
  match heq₀ : t with
  | DCC (Π(Id(Sort m), Λ(_a, _b, `(_c, β))) α) =>
    if heq₁ : t = (DCC (α : Sort m) → β) then
      .some ⟨α, β, m, ⟨by rw [← heq₀, heq₁]⟩⟩
    else
      .none
  | _ => .none
```

Our `Judge` relation models the inference rules (3.3). We have omitted the individual types and corresponding constructors for brevity, but give the constructor for dependent Curry (13), App (18), Π (2) and application as examples. See the appendix (A) for the full formalization.

```
@[dcc_judge] abbrev curr.type (α β γ : Expr) (m : Level) : Expr :=
  DCC (Π(Id(Sort m), `(Sort m, Π(β, Λ(α, β, γ)))) α) -- weaken α, keep x

@[dcc_judge, grind →, grind .] abbrev App.type (f_α f_β δ : Expr) (o : Level) : Expr :=
  let tβ_coup := DCC Λ(δ, f_α, Π(f_α, f_β) ∘ Fstc(f_α))
  DCC (Π(Id(Sort o), `(Sort o, Π(Σ(tβ_coup), Subst(f_β, Fst(tβ_coup))))) δ)

@[dcc_judge] abbrev Pi.type (α : Expr) (m o : Level) : Expr :=
  DCC (α : Sort m) → Sort o

@[dcc_judge] abbrev Pi.f_β.type (f_α α : Expr) (m o : Level) :=
  (DCC (Π(Id(Sort m), `(Sort m, Π(f_α, Λ(α, f_α, `(Coup(f_α), Sort o)))))) α))

@[dcc_judge] abbrev curr.β.type (α : Expr) (m n : Level) : Expr :=
  DCC (α : Sort m) → Sort n

inductive Judge : Expr → Expr → Prop where
  | curr   : Judge β (curr.β.type α m n)
    → Judge x (DCC (Π(f_α, γ) x_α))
    → Reduces (DCC Coup(β)) τα
    → Reduces (DCC (f_α x_α)) τα
    → Reduces (curr.type α β (DCC (γ x_α)) m) τ
    → Judge (DCC Λ(α, β, x)) τ
  | pi     : Judge f_α (arrow α (DCC Sort n) m)
    → Judge f_β (Pi.f_β.type f_α α m o)
    → Reduces (Pi.type α m o) τ
    → Judge (DCC Π(f_α, f_β)) τ
  | appl   : Judge f_α (arrow δ (DCC Sort m) o)
    → Judge f_β (Pi.f_β.type f_α δ o n)
    → Reduces (App.type f_α f_β δ o) τ
    → Judge (DCC App(f_α, f_β)) τ
  | app    : Judge f (DCC (Π(f_α, f_β) x_α))
    → Reduces (DCC (f_α x_α)) α
    → Judge x α
    → Reduces (DCC (f_β x_α x)) τ
    → Judge (DCC (f x)) τ
  | sort   : Judge (DCC Sort m) (DCC Sort m.succ)
  | top    : Judge (DCC ⊤) Top.type
  | bot    : Judge (DCC ⊥) Bot.type
  | tintro : Judge (DCC *) Tintro.type
```

## 5.1 Type Inference Algorithm

Using our terminating affine and $\eta$ reduction functions, we define an `infer` function with soundness. We give only the cases described in our `Judge` illustration, though the rest are available in (A). Pattern matching our types is somewhat obtuse, primarily due to the combinators having several points. Hence, we make use of helper functions to pattern match against them and provide equality proofs. For example, the helper function for pattern-matching arrows and the type of $\beta$ in $\Lambda$ are defined as such:

```
@[dcc_judge] def with_arr (h : Option ((t : Expr) × PULift.{1, 0} (Judge e t))) :
  Option ((τ α β : Expr) × (m : Level) × PULift.{1, 0} (Judge e τ) × PULift.{1, 0} (τ = arrow
  ↪  α β m)) :=
    match h with
    | .some ⟨t, h⟩ => match arrow.matcher t with
      | .some ⟨α, β, m, h_arr⟩ => .some ⟨t, α, β, m, h, h_arr⟩
      | _ => .none
    | _ => .none

@[dcc_judge, inline] abbrev curr.β.type.matcher (t : Expr) : Option
  ((α : Expr) × (m n : Level) × (PULift.{1, 0} (t = curr.β.type α m n))) :=
```

```
  match arrow.matcher t with
  | .some ⟨α, DCC Sort n, m, ⟨h_arr⟩⟩ =>
    .some ⟨α, m, n, ⟨by rw [h_arr]⟩⟩
  | _ => .none

@[dcc_judge] def with_curr_β_type (h : Option ((t : Expr) × PULift.{1, 0} (Judge e t))) :
  Option ((τ : Expr) × (α : Expr) × (m n : Level) × PULift.{1, 0} (Judge e τ) × (PULift.{1, 0}
  ↪  (τ = curr.β.type α m n))) :=
  match h with
  | .some ⟨t, h⟩ => match curr.β.type.matcher t with
    | .some ⟨α, m, n, h₂⟩ => .some ⟨t, α, m, n, h, h₂⟩
    | _ => .none
  | _ => none
```

Inference is computed like such:

```
def infer (e : Expr) : Option ((t : Expr) × PULift.{1, 0} (Judge e t)) :=
  match e with
  | DCC ⊤ => .some ⟨_, ⟨.top⟩⟩
  | DCC ⊥ => .some ⟨_, ⟨.bot⟩⟩
  | DCC * => .some ⟨_, ⟨.tintro⟩⟩
  | DCC Sort _m => .some ⟨_, ⟨.sort⟩⟩
  | DCC Λ(α, β, x) =>
    match with_curr_β_type (infer β) with
    | .some ⟨_, α₂, m, n, ⟨htβ⟩, ⟨harrβ⟩⟩ =>
      match infer x with
      | .some ⟨DCC (Π(f_α, _y) x_α), ⟨ht₂⟩⟩ =>
        if heq : reduce_all (DCC (f_α x_α)) = reduce_all (DCC Coup(β)) then
          if heq₂ : α = α₂ then
            .some ⟨_, ⟨.curr (heq₂ ▸ harrβ ▸ htβ) ht₂ (reduce_reduces heq.symm)
              (reduce_reduces rfl)
              (reduce_reduces rfl)⟩⟩
          else .none
        else .none
      | _ => .none
    | _ => .none
  | DCC App(f_α, f_β) =>
    match with_arr (infer f_α) with
    | .some ⟨_τ₁, δ, DCC Sort m, o, ⟨ht₁⟩, ⟨harr⟩⟩ =>
      match (infer f_β) with
      | .some ⟨DCC (Π(_a, `(_b, Π(_c, Λ(_d, _e, `(_f, Sort n))))) _g), ⟨ht₂⟩⟩ =>
        if heqa : (DCC (Π(_a, `(_b, Π(_c, Λ(_d, _e, `(_f, Sort n))))) _g)) = (Pi.f_β.type f_α
        ↪  δ o n) then
          .some ⟨_, ⟨.appl (harr ▸ ht₁) (heqa ▸ ht₂) (reduce_reduces rfl)⟩⟩
        else .none
      | _ => .none
    | _ => .none
  | DCC Π(f_α, f_β) =>
    match with_arr (infer f_α) with
    | .some ⟨_τ₁, α, DCC Sort _n, m, ⟨ht₁⟩, ⟨harr⟩⟩ =>
      match infer f_β with
      | .some ⟨DCC (Π(_a, `(_b, Π(_c, Λ(_d, _e, `(_f, Sort o))))) _g), ⟨ht₂⟩⟩ =>
        if heq : (DCC (Π(_a, `(_b, Π(_c, Λ(_d, _e, `(_f, Sort o))))) _g)) = Pi.f_β.type f_α α
        ↪  m o then
          .some ⟨_, ⟨.pi (harr ▸ ht₁) (heq ▸ ht₂) (reduce_reduces rfl)⟩⟩
        else .none
      | _ => .none
    | _ => .none
  | DCC (f x) =>
    match infer f, infer x with
    | .some ⟨DCC (Π(f_α, _f_β) x_α), ⟨ht₁⟩⟩, .some ⟨τ₂', ⟨ht₂⟩⟩ =>
      if heq : reduce_all (DCC (f_α x_α)) = τ₂' then
        .some ⟨_, ⟨.app ht₁ (reduce_reduces rfl) (by rw [heq]; exact ht₂) (reduce_reduces
        ↪  rfl)⟩⟩
      else .none
    | _, _ => .none
  | _ => sorry -- remaining cases omitted for brevity
```

Avoiding a conversion constructor proves to be quite advantageous, since neither confluence or induction are necessary to prove useful properties. Furthermore, since we have sound and complete type reduction, completeness is provable.

**Theorem 16.** *For all $e \in Expr$ and $t \in Expr$, if $h$ is a proof that $e : t$, then* `infer e = some (t, h)`.

*Proof.* By tactic automation using our type-class hierarchy and completeness and soundness proofs.

```
theorem Judge.complete (h : Judge e t) : infer e = .some ⟨t, ⟨h⟩⟩ := by
  induction h <;> (
    dsimp only [infer]
    try (simp_all [dcc_judge, dcc_reduce]; aesop (rule_sets := [DccReduce])))
```

□

### 5.2 Judgmental Decidability

Using our proof of completeness and soundness proof produced by `infer`, we demonstrate judgmental decidability.

**Theorem 17.** *For all $e \in Expr$ and $t \in Expr$, then $e : t \vee \neg e : t$.*

*Proof.*

```
instance : Decidable (Judge e t) :=
  if h : (infer e).isSome then
    if heq : ((infer e).get h).fst = t then
      .isTrue (heq ▸ ((infer e).get h).snd.down)
    else
      isFalse (fun h => by simp [h.complete] at *)
  else
    isFalse (fun h => by simp [h.complete] at *)
```

□

## 6 Future Work

We hope to use our inference and reduction machinery to mechanize a proof of strong normalization of the entire calculus in the future. That is, a proof of strong normalization for the calculus including the pair combinator, which is not affine. We have also made significant efforts on compiling dependent $\lambda$-calculus to our combinators, which we hope to finish soon.

## 7 Conclusions

Our formalization gives an extension of the simply-typed categorical combinators [4, p. 220] to a dependently-typed setting. By mechanizing the theory in Lean 4, we provide several guarantees about the logic. Most importantly, we demonstrate a mechanism for deciding type inference in the calculus. We prove strong normalization of an affine restriction on the categorical combinators and $\eta$-reduction, and use this property to prove soundness and completeness of an inference function with respect to typing judgments. We also demonstrate a useful technique to avoid conversion in the typing relation, given we have decidable $\beta$ and $\eta$-reduction.

Our proof of judgmental decidability is made possible in principle by the observation that the types of the dependent categorical combinators are purely affine. This fact is not true of equivalent systems, like the dependent $SK$ combinators.

Finally, since the categorical combinators are well-known to be a performant compilation target, we believe that our verified implementation could have ramifications towards the efficient execution of dependent type inference in a purely contextless way.

## Acknowledgements

To Jake, for teaching me lambda-calculus.

## A Supplementary Materials

**Lean 4 formalization repository:** https://github.com/lexzaiello/dcc
**Omitted inference cases:** https://github.com/lexzaiello/DCC/blob/master/Dcc/Judge.lean
**Omitted reduction cases:** https://github.com/lexzaiello/DCC/blob/master/Dcc/Aff/Red.lean

# References


[1] Thorsten Altenkirch et al. "Combinatory Logic and Lambda Calculus Are Equal, Algebraically". In: *International Conference on Formal Structures for Computation and Deduction*. 2023. URL: https://api.semanticscholar.org/CorpusID:259298739.

[2] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[3] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[4] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.

[5] Maxwell Herman Alexander Newman. "On theories with a combinatorial definition of" equivalence"". In: *Annals of mathematics* 43.2 (1942), pp. 223–243.

[6] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages (Prentice-Hall International Series in Computer Science)*. USA: Prentice-Hall, Inc., 1987. ISBN: 013453333X.

[7] Eike Ritter. *Categorical combinators for the calculus of constructions*. Tech. rep. University of Cambridge, Computer Laboratory, 1990.

[8] R. A. G. Seely. "Locally cartesian closed categories and type theory". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 95.1 (1984), pp. 33–48. DOI: 10.1017/S0305004100061284.